CLA 4 — ML Model Deployment Strategies and Model Serving Using FastAPI

1. Objective

To develop a machine learning classification model using the Titanic dataset, demonstrate online, batch, and streaming inference, and deploy the trained model through a REST API using FastAPI.

2. Dataset

The Titanic – Machine Learning from Disaster dataset was obtained from Kaggle.

Dataset link: https://www.kaggle.com/competitions/titanic/data

The train.csv file was used for preprocessing, training, and evaluation. The target variable was Survived, where 0 represents did not survive and 1 represents survived.

3. Model Training

A Random Forest classification model was trained using selected passenger features. Missing numerical values were replaced with median values, and missing categorical values were replaced with the most frequent values. Categorical features were encoded using One-Hot Encoding.

The dataset was split into training and testing portions. The complete preprocessing and classification pipeline was saved as model.pkl using Joblib.

Model accuracy:

Model Accuracy: 0.8156

Model Accuracy Percentage: 81.56%

4. Online Inference

Online inference was performed by loading the saved model and predicting the survival outcome for one held-out passenger record. The passenger details and predicted class were displayed in the terminal.

5. Batch Inference

Batch inference was performed on 20 held-out passenger records. The model generated predictions for all records together, and the results were saved in batch_predictions.csv.

6. Streaming Inference

Streaming inference was simulated using 10 held-out passenger records. Each record was processed individually with a one-second delay between consecutive records. The prediction for each incoming record was displayed in the terminal.

7. FastAPI Model Serving

The trained pipeline was deployed using FastAPI through the following endpoints:

- **GET /health:** Checks the API status and confirms that the model is loaded.
- **POST /predict:** Accepts passenger details, validates the input using Pydantic, and returns a survival prediction.

The endpoints were tested using Swagger UI at /docs. A valid request returned HTTP 200, while invalid passenger input returned HTTP 422. The prediction endpoint was also tested using a Python client, client.py.

8. Observations

8.1 How does Online Inference differ from Batch Inference?

Online inference generates a prediction for an individual input when it is received. Batch inference processes multiple records together in one operation. Online inference is useful for individual requests, while batch inference is suitable for bulk predictions.

8.2 What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes incoming records continuously, often one at a time as they arrive. Batch inference processes a fixed collection of records together. Streaming supports ongoing processing, while batch inference is useful for scheduled or bulk processing.

8.3 Why is FastAPI used for model serving?

FastAPI provides a convenient way to expose a machine learning model through REST API endpoints. It supports request validation with Pydantic, automatic interactive documentation through Swagger UI, and HTTP-based communication with applications.

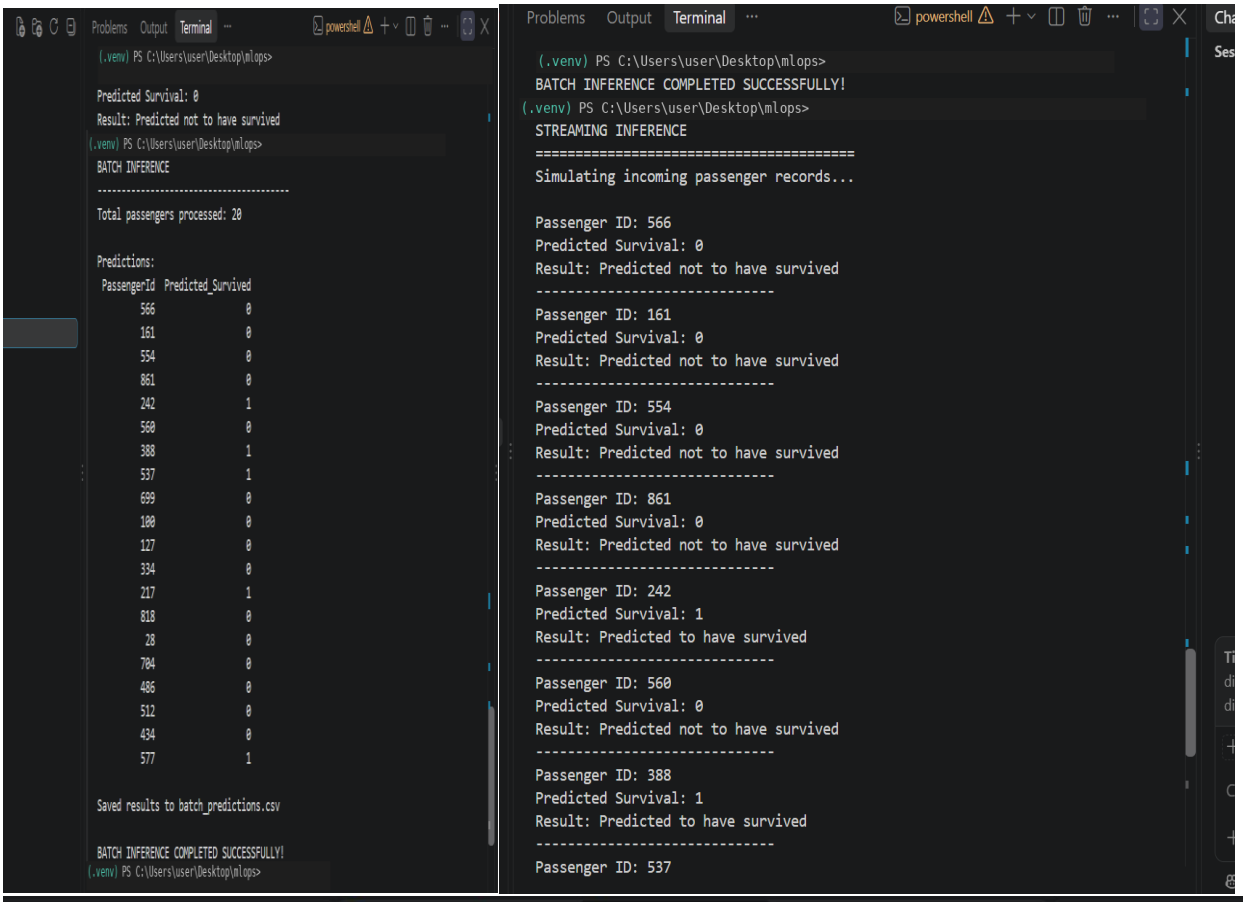
8.4 Why should the same trained model be reused for all inference strategies?

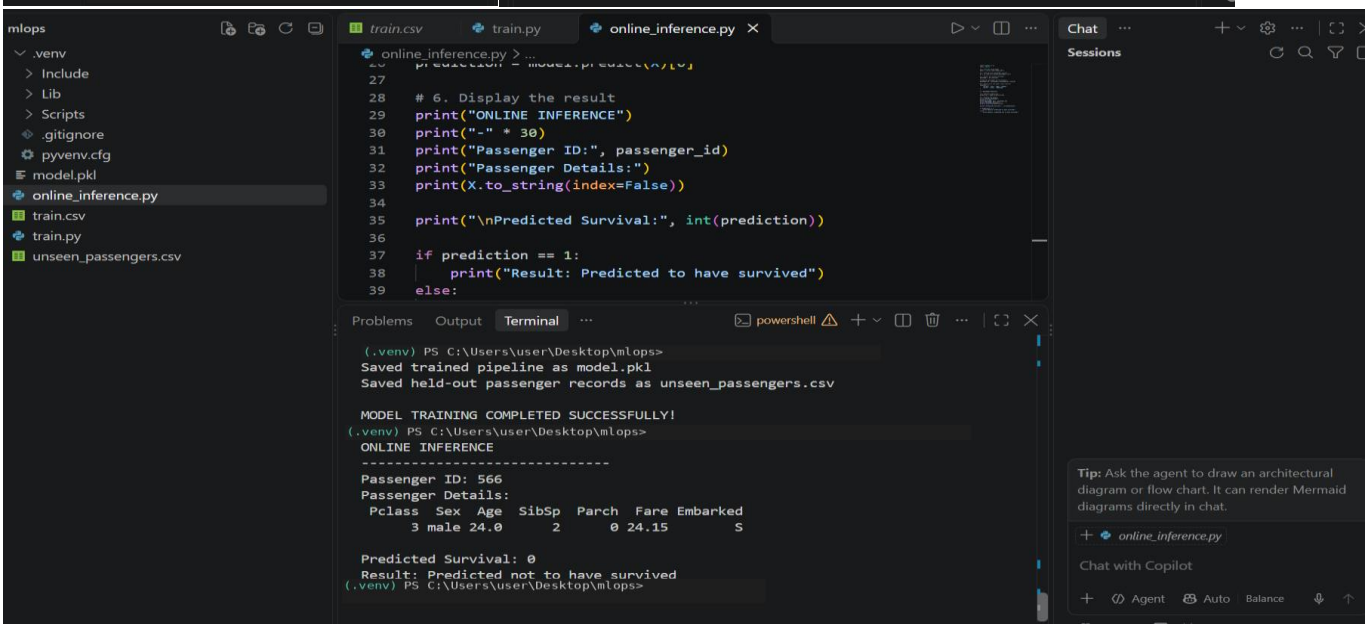
Reusing the same trained model ensures consistent predictions across online, batch, streaming, and API-based inference. It avoids unnecessary retraining, reduces computational overhead, and enables a fair comparison of the inference strategies.

9. Conclusion

The assignment demonstrates the machine learning deployment workflow, including data preprocessing, model training, evaluation, saving the trained pipeline, three inference strategies, and REST API-based model serving using FastAPI. The Python client and Swagger UI were used to test the deployed prediction service.

10. Evidence





client.py > ...

```
        print("-" * 35)
        print("HTTP Status:", response.status_code)

        response.raise_for_status()

        print("Passenger Details:", passenger)
        print("Prediction Response:")
        print(response.json())

        print("\nCLIENT TEST COMPLETED SUCCESSFULLY!")

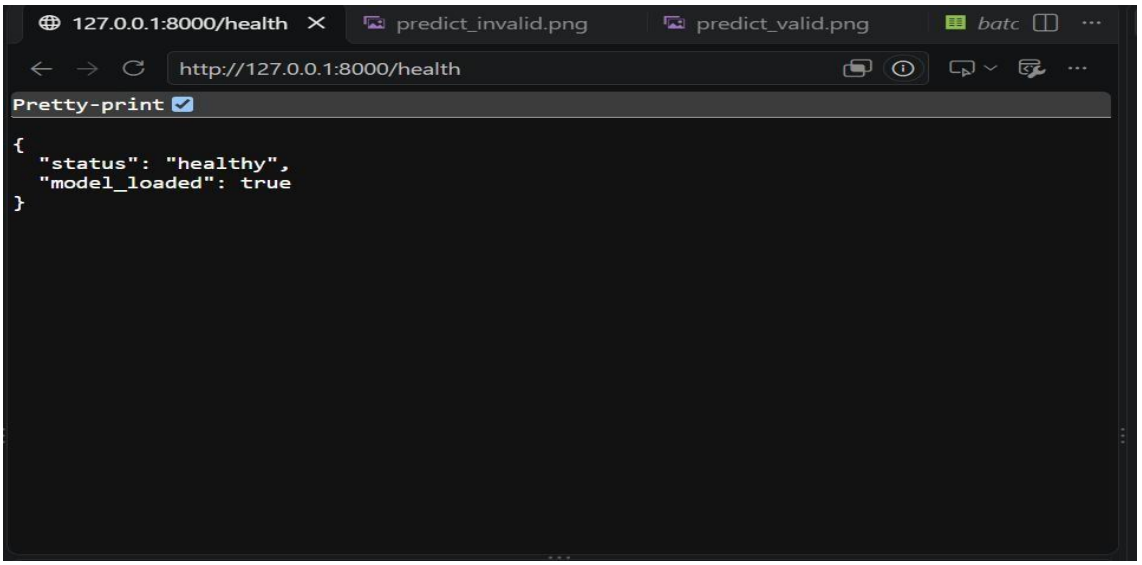
    except requests.exceptions.RequestException as error:
        print("API request failed:", error)
        print("Check that the FastAPI server is running.")
```

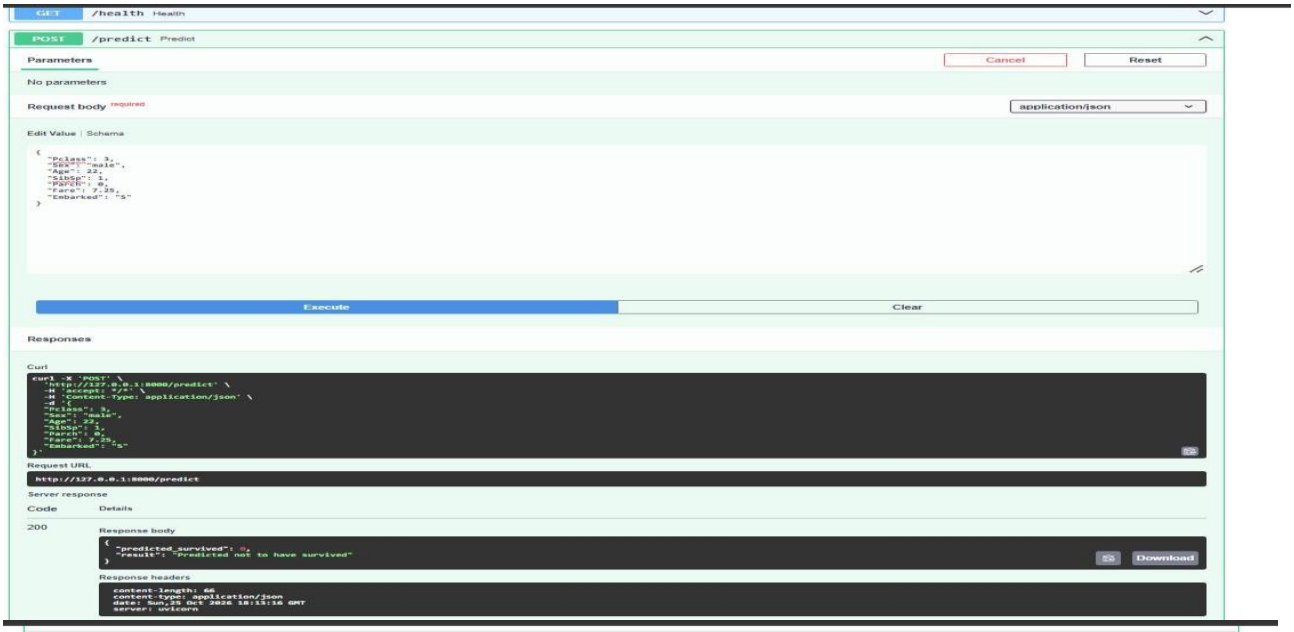
Problems Output Terminal ... powershell

```
C:\Users\user\Desktop\mlops>

 v\Scripts\Activate.ps1)
(.venv) PS C:\Users\user\Desktop\mlops>
 FASTAPI PYTHON CLIENT
 ----------------------------------
 HTTP Status: 200
 Passenger Details: {'Pclass': 3, 'Sex': 'male', 'Age': 22, 'SibSp': 1, 'Par
 ch': 0, 'Fare': 7.25, 'Embarked': 'S'}
 Prediction Response:
 {'predicted_survived': 0, 'result': 'Predicted not to have survived'}

 CLIENT TEST COMPLETED SUCCESSFULLY!
(.venv) PS C:\Users\user\Desktop\mlops>
```





Edit Value | Schema
{
"Pclass": 5,
"Sex": "unknown",
"Age": 22,
"SibSp": 1,
"Parch": 0,
"Fare": 7.25,
"Embarked": "S"
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/predict' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"Pclass": 5,
"Sex": "unknown",
"Age": 22,
"SibSp": 1,
"Parch": 0,
"Fare": 7.25,
"Embarked": "S"
}'
Request URL
http://127.0.0.1:8000/predict
Server response
Code
Details
422
Error: Unprocessable Content
Response body

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "Pclass"
      ],
      "msg": "Input should be less than or equal to 3",
      "input": 5,
      "ctx": {
        "le": 3
      }
    },
    {
      "type": "string_pattern_mismatch",
      "loc": [
        "body",
        "Sex"
      ],
      "msg": "String should match pattern '^(male|female)$'",
      "input": "unknown",
      "ctx": {
        "pattern": "^(male|female)$"
      }
    }
  ]
}
```

Download